

Laravel Backend Developer — Interview Preparation Guide

Tailored for: Saurabh Kumar Jha | Target Role: PHP Laravel Backend Developer
(Dwarka Sector 14, New Delhi)

Section 1: PHP & Laravel Core

Q1. Explain the request lifecycle in Laravel. A: A request enters through `public/index.php`, which boots the Laravel application via the kernel. It passes through global middleware (e.g., CORS, CSRF), gets matched to a route in `routes/web.php` or `routes/api.php`, then route-specific middleware runs (auth, throttling), the controller method executes (often calling a service or repository layer), and finally a response — view, JSON, or redirect — is returned back through the middleware stack to the browser.

Q2. What is MVC architecture and how does Laravel implement it? A: MVC separates an app into Model (data and business logic, e.g., Eloquent models), View (presentation, Blade templates), and Controller (handles requests, coordinates between Model and View). Laravel enforces this structure by convention — controllers in `app/Http/Controllers`, models in `app/Models`, views in `resources/views`. *Tie-in: mention your CCPD/DIAC case management systems as MVC-structured Laravel/CodeIgniter apps.*

Q3. What's the difference between `hasMany`, `belongsTo`, `belongsToMany`, and `hasManyThrough` in Eloquent? A: `hasMany`/`belongsTo` define one-to-many and its inverse (e.g., a Case has many Documents). `belongsToMany` defines many-to-many via a pivot table (e.g., Users and Roles). `hasManyThrough` accesses a distant relation through an intermediate model (e.g., a Department has many Cases through Users).

Q4. What is the N+1 query problem and how do you solve it? A: It happens when you loop over a collection and lazily access a relationship inside the loop, triggering one query per record. Fix with eager loading: `Model::with('relation')->get()` instead of querying inside a loop. *Tie-in: mention this directly relates to your "optimized SQL queries and backend services" line at Kreate Technology — be ready to describe a real N+1 fix you made.*

Q5. What are Laravel Service Providers? A: Bootstrapping classes where you bind things into the service container, register bindings, event listeners, or configuration. Every package and core Laravel feature is wired up through providers in `config/app.php` or auto-discovery.

Q6. Explain dependency injection and the service container in Laravel. A: Laravel's IoC container automatically resolves class dependencies. If a controller's constructor type-hints an interface or class, Laravel resolves and injects an instance automatically, which makes code more testable and decoupled.

Q7. What's the difference between `Laravel Migrations` and seeders? A: Migrations define and version-control database schema changes (creating/altering tables). Seeders populate the database with sample or default data, often used in conjunction with factories for testing.

Q8. What is middleware and give an example you've used. A: Middleware filters HTTP requests before they hit the controller (or after, on the way out). Built-in examples: `auth`, `throttle`, `verified`. You can also write custom middleware, e.g., for role-based access control — directly relevant since your CCPD project mentions role-based access.

Section 2: REST APIs

Q9. How do you design a RESTful API in Laravel? A: Use resource controllers (`php artisan make:controller --resource`), map HTTP verbs to actions (GET/POST/PUT/DELETE), return consistent JSON structures (often via API Resources — `php artisan make:resource`), use proper HTTP status codes, and version routes (`/api/v1/...`).

Q10. What's the difference between PUT and PATCH? A: PUT replaces the entire resource; PATCH applies a partial update to specific fields.

Q11. How do you handle API authentication in Laravel? A: Laravel Sanctum for SPA/token-based auth (lightweight, good for first-party apps), or Passport for full OAuth2 (third-party apps, scopes). Sanctum issues personal access tokens; the `auth:sanctum` middleware protects routes.

Q12. How do you version and document your APIs? A: Versioning through route prefixes (`/api/v1/`) or headers. Documentation via tools like Swagger/OpenAPI or Postman collections, kept in sync with the codebase.

Q13. How would you secure a public-facing API from abuse? A: Rate limiting (`throttle` middleware), input validation (Form Requests), authentication tokens, CORS configuration, and avoiding exposing internal IDs/sensitive data in responses.

Be ready to walk through the API design for your CCPD or DIAC case filing system as a real example here.

Section 3: Laravel Queues & Cron Jobs (you said you have experience — be specific in the actual interview)

Q14. What are Laravel Queues and why use them? A: Queues let you defer time-consuming tasks (emails, notifications, report generation, third-party API calls) to be

processed asynchronously by a background worker, instead of blocking the HTTP response. Configured via `config/queue.php`, run with `php artisan queue:work`.

Q15. What queue drivers have you used (database, Redis, SQS)? A: Be ready to name the actual driver you used. Database driver is simplest (jobs stored in a `jobs` table); Redis is faster and supports more features; SQS is for cloud/AWS setups.

Q16. How do you handle failed jobs? A: Laravel logs failed jobs to a `failed_jobs` table (via `php artisan queue:failed-table` migration). You can retry with `php artisan queue:retry`, and define a `failed()` method on the Job class for custom failure handling (e.g., alerting).

Q17. What's the difference between a Job and an Event Listener in Laravel? A: A Job is a discrete unit of work dispatched to a queue. An Event Listener responds to an Event being fired (can also be queued by implementing `ShouldQueue`). Jobs are typically used for tasks; Events/Listeners for decoupled reactions to things happening in the app.

Q18. How do you schedule recurring tasks (cron) in Laravel? A: Define scheduled tasks in `app/Console/Kernel.php` (or `routes/console.php` in newer versions) using the `Schedule` facade — e.g., `$schedule->command('emails:send')->daily()`. A single cron entry (`* * * * *` `php artisan schedule:run`) on the server triggers Laravel's scheduler every minute, which then runs due tasks.

Prep tip: Have one real example ready — what specific recurring task or background job did you build, on which project, and what problem did it solve? This is a required skill on the JD, so expect a direct, specific follow-up question.

Section 4: Database (MySQL)

Q19. How do you optimize a slow MySQL query? A: Use `EXPLAIN` to inspect the query plan, add indexes on frequently filtered/joined columns, avoid `SELECT *`, avoid N+1 patterns, denormalize selectively if needed, and check for missing composite indexes on multi-column WHERE clauses.

Q20. What's the difference between INNER JOIN, LEFT JOIN, and RIGHT JOIN? A: INNER JOIN returns only matching rows in both tables. LEFT JOIN returns all rows from the left table plus matches from the right (NULLs where no match). RIGHT JOIN is the mirror of LEFT JOIN.

Q21. What's database normalization, and have you ever denormalized for performance? A: Normalization organizes data to reduce redundancy (1NF, 2NF, 3NF). Denormalization intentionally introduces redundancy (e.g., a cached count column) to avoid expensive joins/aggregations at read time, trading storage for speed.

Q22. How do you design a schema for a workflow/case-tracking system? A: Talk through your actual CCPD/DIAC schema — cases table, status/workflow tables, documents table with foreign keys, audit/history table for tracking status changes, users/roles tables for access control.

Section 5: Authentication & Security

Q23. How does Laravel handle CSRF protection? A: Laravel auto-generates a CSRF token per session, included in forms via `@csrf` Blade directive, verified by the `VerifyCsrfToken` middleware on state-changing requests.

Q24. How do you implement role-based access control (RBAC)? A: Common approaches: Laravel's built-in Gates/Policies for fine-grained authorization, or packages like Spatie's `laravel-permission` for role/permission management. You've implemented "role-based access" on CCPD — be ready to explain exactly how (custom middleware, Gates, or a package).

Q25. How do you securely store and validate user passwords? A: Laravel hashes passwords with bcrypt (or Argon2) via the `Hash` facade automatically — never store plaintext. Validation enforced through Form Request rules (`min`, `confirmed`, etc.).

Q26. What's the difference between authentication and authorization? A: Authentication verifies who the user is (login). Authorization determines what they're allowed to do (permissions/roles) — directly relevant to your case management role-based access work.

Section 6: Payment Gateway / Third-Party Integration (you mentioned experience — prep a specific story)

Q27. Walk me through how you integrated a payment gateway. A: Structure your answer: (1) which gateway and why, (2) how you handled the API keys/secrets securely (`.env`, not hardcoded), (3) how you handled the payment flow (initiate → redirect/callback → webhook verification), (4) how you handled failures/retries and kept records in your database, (5) any security considerations (signature verification, idempotency).

Q28. How do you handle webhook callbacks securely? A: Verify the webhook signature against a shared secret, use HTTPS, make the handler idempotent (in case of duplicate calls), and log all incoming events for audit/debugging.

Q29. What's the difference between synchronous and asynchronous payment confirmation? A: Synchronous — the gateway returns success/failure directly on the request (immediate redirect). Asynchronous — the gateway separately sends a

webhook/IPN later confirming the final status, which is more reliable since users can close the browser before redirect completes.

Section 7: WhatsApp Business API / Linux (mention whichever you actually used)

Q30. How does WhatsApp Business API integration typically work? A: Through Meta's Cloud API or a BSP (Business Solution Provider) like Twilio/Gupshup — you send template messages or session messages via REST calls with an access token, and receive incoming messages/status updates via configured webhooks.

Q31. What basic Linux server tasks have you done (deployment, SSH, permissions)? A: Be ready to mention specifics: deploying a Laravel app (composer install, `.env` setup, storage permissions, queue worker via supervisor), basic SSH usage, checking logs (`tail -f storage/logs/laravel.log`), restarting services.

Section 8: Debugging & Problem Solving

Q32. Tell me about a production bug you diagnosed and fixed. A: Use your Capgemini Health Insurance platform experience — describe the symptom, how you isolated the root cause (logs, debugging tools, reproducing locally), the fix, and how you verified it didn't break anything else.

Q33. How do you debug a Laravel application in production without bringing it down? A: Check `storage/logs/laravel.log`, use logging strategically (`Log::info/error`), tools like Laravel Telescope (non-production usually) or Sentry/Bugsnag for error tracking, and reproduce safely in staging before patching production.

Q34. How do you approach a bug report with no clear reproduction steps? A: Gather context (logs, timestamps, affected user/data), check recent deployments/changes around that time, narrow down with logging, and isolate the smallest reproducible case.

Section 9: Behavioral / Role-Fit Questions

Q35. Why are you interested in this role? A: Tie it to genuine overlap: backend Laravel development, REST APIs, your experience building case management/workflow systems (which maps to "logistics and complex workflow" mentioned in the JD), and your interest in working on scalable backend systems.

Q36. Tell me about the CCPD Court Case Management System — what was your specific contribution? A: Be ready with a clear, structured answer: your role, the backend APIs you designed, the database schema decisions, and the performance optimizations you made.

Q37. Why are you looking to move from your current/most recent role? A: Keep this professional and forward-looking — focus on seeking new challenges, broader scope, or growth, not anything negative about past employers.

Q38. How do you handle conflicting priorities or tight deadlines? A: Give a structured example — how you triaged, communicated with stakeholders, and delivered.

Q39. Describe a time you collaborated with frontend developers or product teams. A: Reference your work across CCPD/DIAC/MBKS portals — how you aligned on API contracts, handled change requests, and communicated blockers.

Section 10: Possible Live/Take-Home Task — What to Expect

Given the JD, a practical round may ask you to:

- Build a simple REST API endpoint (e.g., CRUD for a "case" or "order" resource) with validation and proper status codes
- Demonstrate a queued job (e.g., dispatch a job that simulates sending a notification)
- Write an Eloquent query involving a relationship and explain how you'd avoid N+1
- Debug a small piece of broken Laravel code they hand you

Prep action: Before the interview, build (or review) a small Laravel project locally that includes a REST API resource, one queued job, and one Eloquent relationship with eager loading — so you can speak about working code, not just theory.

Final Prep Checklist

- ☐ Prepare your specific queue/cron example (what task, what problem it solved)
- ☐ Prepare your specific payment gateway example (which one, what flow)
- ☐ Prepare your specific WhatsApp API or Linux example
- ☐ Prepare a 2-minute walkthrough of CCPD project end-to-end
- ☐ Review N+1 queries and eager loading — likely to come up
- ☐ Review Laravel Sanctum vs Passport
- ☐ Practice explaining role-based access implementation in plain terms
- ☐ Have one strong "bug I fixed" story ready (Capgemini experience)